

ReverseBids Advanced Engine

Phase-2 Build Report — Inclusions, Exclusions, Limitations & Work Estimate

Result

The full Phase-2 specification was built in one session: multi-seller partial allocation (L1-L12, pay-as-bid, rollover), quality-normalized pricing (protein 62-67), negotiation/match flow, blind identities, lots, reservation→confirm allocation, indent + email generation, audit trail, and a FastAPI surface — all verified by 18 passing tests and a runnable 1000T simulation that allocates 300/500/200.

1. Inclusions (delivered & tested)

Capability	Status	Where
RFQ lifecycle + admin approvals + state machine	Done	engine.py
Partial & variable-quantity bidding (300/400/500/600/700T)	Done	engine.submit_bid
L1-L12 ranking with pay-as-bid allocation + rollover	Done	allocation.py
Quality-normalized pricing (protein 62-67, configurable base)	Done	pricing.py
Negotiation / match flow (offer, accept/reject, expiry, rule)	Done	negotiation.py / engine
Blind identities (S-XXXX aliases), no competitor leak	Done	aliasing.py / engine views
Lots / split RFQ, per-lot ranking & allocation	Done	engine (lot_id)
Reservation → admin confirm → final allocation + statuses	Done	engine reservations
Indent document generation (admin vs masked copies)	Done	emailer.render_indent
Email abstraction + fake adapter + auto-send on confirm	Done	emailer.py
Decimal money/qty, injectable clock, per-RFQ locks, anti-snipe	Done	throughout
Privacy-safe audit trail	Done	engine._audit
FastAPI endpoints for all of the above	Done	api.py
Test suite (16 scenarios + 2 HTTP) + 1000T demo	Done — 18/18 pass	tests/, run_demo.py

2. Exclusions (intentionally out of this engine)

Deliberately NOT in this engine	Why / where it belongs
Database persistence (PostgreSQL)	In-memory store behind an interface; Phase 3 swap.
Real SMTP / WhatsApp sending	Fake adapter only; real adapters are a config swap (Phase 3).
AuthN/AuthZ (who is admin/buyer/seller)	Built in the earlier Phase-2 auth module; wire it onto these endpoints.
Frontend dashboards	Earlier Phase-2 web layer; not re-bundled here.
Real-time WebSocket push	REST polling for MVP; sockets are Phase 3.
Payments / invoicing / GST documents	Out of scope for the auction engine.
Freight rate calculator	Freight is an input per bid for now.

3. Limitations (known, with mitigations)

Limitation	Impact / mitigation
State lives in memory	Restart loses data — fine for MVP/demo; move to DB before production.
No background scheduler	Reservation/negotiation expiry is evaluated on access, not by a timer job.
Single-process locking	Per-RFQ locks protect one process; multi-instance needs DB row locks/queue.
Pay-as-bid only	Clearing-price model not implemented (was an explicit business choice).
Protein normalization is linear	Uses base/actual ratio; real contracts may use stepped premia — configurable later.
Email body is plain text	No HTML/PDF indent rendering yet (renderer is swappable).

4. Key Assumptions & Decisions

- **Allocation order:** global rank by normalized effective price ascending (lowest = L1); ties by earliest arrival.
- **Pay-as-bid:** each seller allocated at their own raw price (not a single clearing price) — your confirmed choice.
- **Protein normalization:** $\text{normalized_price} = \text{raw} \times (\text{base} / \text{actual})$; base default 62; integers 62–67 only; freight added after for the ranking key.
- **Rollover:** sequential down ranks (L1 exhausted → L2 becomes active L1), capped at L12; shortfall → partial fulfilment for negotiation.
- **Admin confirmation mandatory:** reservation → admin confirm → final allocation; over-allocation is blocked.
- **Blind auction:** sellers see only their own alias + masked benchmark; buyers see aliases; only admin sees real identities; audit log is alias-only.

5. Lean MVP vs Phase 3

Everything in Section 1 is the Lean MVP and runs today. Section 2 items are Phase 3: they are deliberately swappable (the store is behind an interface; email is behind a provider; auth/frontend already exist as separate Phase-2 layers from the earlier build).

6. Work Done Today

Item	Estimate
Domain models (13 entities) + enums	done today
Pricing (protein normalization) + tests	done today
L1-L12 allocation + rollover + tests	done today
Negotiation/match flow + expiry + tests	done today
Aliasing / blind views + leak tests	done today
Lots / multi-lot allocation + tests	done today
Reservation → confirm → allocation + over-alloc guard	done today
Indent + email abstraction + auto-send	done today
FastAPI surface + HTTP tests	done today
1000T runnable simulation	done today
Code PDF + this report + zip	done today

7. Balance Estimate (to production)

Remaining (Phase 3 / productionization)	Rough effort
PostgreSQL persistence behind the store interface	2-3 days
Wire auth (admin/buyer/seller) onto endpoints	1 day
Real SMTP + WhatsApp adapters + templates	1-2 days
WebSocket live updates + frontend integration	3-4 days
Background expiry scheduler + multi-instance locking	2 days
Security hardening, load test, deploy (Docker/VPS)	2-3 days

Honest note on pacing

You asked me not to pause. I completed the entire Phase-2 spec in this one session and verified it (18/18 tests, runnable demo). I can't promise an unlimited single session in general, so the working method remains: deliver in verified, self-contained increments — here, the whole engine is one such increment, complete and runnable. The remaining work above is genuinely separate productionization, best done as its own session.